Lab 7: Computing the Sink Tree with Dijkstra's Algorithm

Computer Networks · Week 8 · Network Layer / Routing

What this lab does

Given a network topology, a weighted graph of routers and the links between them, you compute the sink tree for a chosen destination: the tree of shortest paths from every router to that destination, using Dijkstra's algorithm. You produce three results: the shortest-path cost from each router to the sink, the sink tree itself, and the forwarding table of next hops that each router would use to reach the sink.

Now the shared object is a graph of many links, and the task is choosing routes across it. This is exactly what a link-state router does: each router runs Dijkstra on the whole topology to build its routing table.

You will:

- represent a weighted network topology as a graph;
- implement Dijkstra from the sink to obtain shortest costs and predecessors;
- extract the sink tree and the next-hop forwarding table;
- verify the tree's properties and observe how it changes when a link cost changes.

Setup

Use any C++ compiler or Python 3. Implement the graph and the algorithm yourself; do not use a graph library that provides shortest paths.

Conventions and constants (the shared specification)

- Routers are labelled nodes. Links are undirected and carry a positive integer cost.
- The topology is given as an edge list of triples `(u, v, cost)`.
- The sink is a chosen destination node.
- The cost of a path is the sum of its link costs. The distance from a node to the sink is the cost of the least-cost path.
- Tie-break rule: when a node can reach the sink by two paths of equal cost, choose the one whose next hop (the node's parent in the tree) has the smaller label. This makes the sink tree unique, so your output can be graded exactly.

Background: what a sink tree is

For a fixed destination, consider the least-cost path from every router to that destination. By the optimality principle, if a router lies on the least-cost path from some source to the

destination, then the least-cost path from that router to the destination is the tail of the same route. As a consequence, the set of least-cost paths from all sources to one destination has no cycles and forms a tree rooted at the destination. That tree is the sink tree.

On an undirected graph the sink tree rooted at D is the same tree that Dijkstra produces when run from D, because the least-cost path from any node X to D is the reverse of the least-cost path from D to X. So you compute the sink tree by running Dijkstra once, from the sink. The predecessor that Dijkstra assigns to each node is that node's next hop toward the sink.

The topology (used by every test vector)

Link	Cost
A–B	2
A–C	5
B–C	1
B–D	4
C–E	3
D–E	1
D–F	2
E–F	6

Six routers, A through F. All links are undirected.

Test vectors (your output must match these exactly)

Vector 1, sink = F.

- Cost to F: A = 8, B = 6, C = 6, D = 2, E = 3, F = 0.
- Next hop toward F: A to B, B to D, C to E, D to F, E to D.
- Sink-tree edges (node, parent): (A, B), (B, D), (C, E), (D, F), (E, D).

Vector 2, sink = A.

- Cost to A: A = 0, B = 2, C = 3, D = 6, E = 6, F = 8.
- Next hop toward A: B to A, C to B, D to B, E to C, F to D.
- Sink-tree edges: (B, A), (C, B), (D, B), (E, C), (F, D).

Vector 3, modified topology, sink = F. Raise the cost of link D–F from 2 to 9, leave everything else unchanged.

- Cost to F: A = 12, B = 10, C = 9, D = 7, E = 6, F = 0.
- Next hop toward F: A to B, B to C, C to E, D to E, E to F.
- Sink-tree edges: (A, B), (B, C), (C, E), (D, E), (E, F).

Part A: Represent the topology

What you do: read the edge list and build an adjacency structure for the graph.

Your task:

1. Store the topology as an adjacency structure that, for each router, lists its neighbours and the cost of each link. Remember that each undirected link appears in both endpoints' neighbour lists.
2. As a check, print each router's neighbours with costs and confirm the graph matches the topology table.

Part B: Dijkstra from the sink

What you do: implement Dijkstra's algorithm starting at the sink to obtain the shortest cost and the predecessor for every router.

Background: Dijkstra grows a set of finalised nodes. It repeatedly finalises the unfinalised node of least tentative cost, then relaxes each of its links, lowering a neighbour's tentative cost when a cheaper path through the finalised node is found. Apply the tie-break rule during relaxation so the predecessor is chosen deterministically.

Your task:

1. Implement Dijkstra from the sink, producing the cost to the sink and the predecessor (next hop toward the sink) for every router.
2. Run it for sink F and confirm the costs and next hops match Vector 1.

Part C: Sink tree and forwarding table

What you do: turn the predecessors into the sink tree and the forwarding table.

Your task:

1. List the sink-tree edges as (node, parent) pairs for every router except the sink.
2. Build the forwarding table: for each router, the next hop it sends to in order to reach the sink, and the total cost.
3. Confirm both match Vector 1, then run the whole pipeline for sink A and confirm Vector 2.

Part D: Properties and a rerouting experiment

What you do: check that the result is a valid tree, then change a link cost and account for how the tree changes.

Your task:

1. Verify that the sink tree is a spanning tree of the topology: it has exactly N minus 1 edges, it reaches every router, and it contains no cycle.

2. Raise the cost of link D–F from 2 to 9 and recompute the sink tree for F. Confirm your result matches Vector 3, then list which tree edges changed and explain, using the optimality principle, why traffic that previously reached F through D now reaches it through E.
3. Explain in two or three sentences why, on this undirected graph, the sink tree for a destination (all routers to one destination) is the same tree Dijkstra produces from that destination (one source to all routers), and describe one situation, such as links with different costs in each direction, where the two would differ.

What to submit

- Your source file (C++ or Python) implementing the graph, Dijkstra, the sink tree, and the forwarding table.
- Program output reproducing Vectors 1, 2, and 3 exactly.
- Part D: the spanning-tree checks, the list of changed edges for the modified topology, and the two written explanations.